

CHANDIGARH UNIVERSITY APEX INSTITUTE OF TECHNOLOGY



ASSIGNMENT-2

Subject: Computer Network
Code:24CST-339

Submitted by:
Name: Shreyansh Gupta
UID:24BAI70658
Class:24AIT_KRG-1B

Submitted to:
Mr. Alok Singh Rathore

Q1. IP Address, IPv4 Classes, Subnetting and Supernetting

1. What is an IP Address?

An **IP (Internet Protocol) address** is a unique logical address assigned to a device on a network. It is used to **identify the device and determine where data should be delivered**.

For example:

192.168.1.10

An IPv4 address consists of **32 bits**, divided into **4 octets**, each containing 8 bits.

Example:

192.168.1.10

Binary:

11000000.10101000.00000001.00001010

2. Why do we need classes in IPv4?

IPv4 originally used a **classful addressing system** to divide the huge IPv4 address space into different categories.

Different organizations/networks require different numbers of IP addresses. Classes allowed networks to be categorized according to their size:

- **Class A** → Very large networks
- **Class B** → Medium-sized networks
- **Class C** → Small networks
- **Class D** → Multicasting
- **Class E** → Experimental/reserved

Note: Modern networks primarily use **CIDR**, rather than classful addressing, but IPv4 classes are still important for understanding the fundamentals.

IPv4 Classes

Class	First Octet	Default Mask	Network Bits	Host Bits	Possible Networks	Hosts per Network
A	1–126	255.0.0.0 (/8)	8	24	126	16,777,214
B	128–191	255.255.0.0 (/16)	16	16	16,384	65,534
C	192–223	255.255.255.0 (/24)	24	8	2,097,152	254
D	224–239	—	—	—	Multicast	—
E	240–255	—	—	—	Experimental	—

Why are some addresses subtracted?

For normal networks:

$$\text{Hosts} = 2^H - 2$$

The two addresses are reserved for:

- **Network address**
- **Broadcast address**

For example, Class C has 8 host bits:

$$2^8 - 2 = 254 \text{ hosts}$$

Class A example

10.0.0.0

Default mask:

255.0.0.0

The first 8 bits identify the network and remaining 24 bits identify hosts.

Therefore:

$$2^{24} - 2 = 16,777,214 \text{ hosts}$$

3. Subnetting

Subnetting is the process of dividing one large network into **smaller logical networks called subnets**.

It is done by **borrowing bits from the host portion** and using them as network/subnet bits.

Example

Suppose we have:

192.168.1.0/24

Default:

255.255.255.0

It has:

- 24 network bits
- 8 host bits

Suppose we borrow **2 host bits**.

New prefix:

/26

New subnet mask:

255.255.255.192

Number of subnets:

$$2^2 = 4$$

Hosts per subnet:

$$2^6 - 2 = 62$$

The four subnets are:

Subnet	Network Address	Usable Hosts	Broadcast
--------	-----------------	--------------	-----------

1	192.168.1.0	.1 – .62	.63
2	192.168.1.64	.65 – .126	.127
3	192.168.1.128	.129 – .190	.191
4	192.168.1.192	.193 – .254	.255

Advantages of subnetting

- Reduces network congestion
- Improves network management
- Uses IP addresses more efficiently
- Provides better security/isolation
- Reduces broadcast traffic

4. Supernetting

Supernetting is the opposite concept of subnetting.

It combines **multiple smaller networks into one larger logical network**.

It is also called **route aggregation** or **route summarization**.

Example

Suppose we have:

- 192.168.0.0/24
- 192.168.1.0/24
- 192.168.2.0/24
- 192.168.3.0/24

These four networks can be combined into:

192.168.0.0/22

Why?

Four /24 networks contain:

$4 \times 256 = 1024$ addresses

A /22 network also contains:

$2^{(32-22)} = 1024$ addresses

Difference

Subnetting:

One large network → multiple smaller networks

Supernetting:

Multiple smaller networks → one larger network

2. Link State vs Distance Vector Routing

1. What is Routing?

Routing is the process of selecting a path through a network for transferring packets from a source to a destination.

The goal is generally to find a **least-cost path**.

The cost could represent:

- Number of hops
- Delay
- Bandwidth
- Reliability
- Administrative cost

2. Distance Vector Routing

In **Distance Vector routing**, each router maintains information about:

- Destination
- Distance/cost to destination
- Next hop

Routers periodically exchange their routing information with their **neighboring routers**.

A common algorithm used is **Bellman-Ford**.

Example

Consider:

A ---1--- B ---2--- C

\ /
-----5-----

Costs:

- $A \rightarrow B = 1$
- $B \rightarrow C = 2$
- $A \rightarrow C = 5$

Initially, A may know:

$$A \rightarrow C = 5$$

After receiving information from B:

$$A \rightarrow B + B \rightarrow C$$

$$= 1 + 2$$

$$= 3$$

Therefore:

$$A \rightarrow B \rightarrow C$$

has a cost of **3**, which is less than the direct cost of 5.

So A updates its routing table:

Destination Cost Next Hop

B	1	B
---	---	---

C	3	B
---	---	---

Characteristics

- Information is exchanged with neighbors.
- Routers don't have complete knowledge of the network.
- Uses Bellman-Ford-based calculations.
- Can suffer from **count-to-infinity** problems.
- Example protocol: **RIP**.

3. Link State Routing

In **Link State routing**, every router builds a map/database representing the network topology.

Each router:

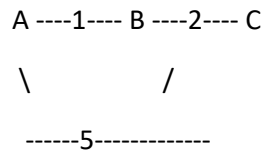
1. Discovers its neighbors.
2. Determines the cost of links.
3. Creates a Link State Advertisement.
4. Floods this information throughout the network.
5. Builds a complete topology.

6. Calculates the shortest path.

A common algorithm is **Dijkstra's Shortest Path First (SPF)** algorithm.

Example

Consider:



Router A knows all the links.

Possible paths from A to C:

Direct:

$A \rightarrow C = 5$

Through B:

$A \rightarrow B \rightarrow C$

$= 1 + 2 = 3$

Therefore Dijkstra's algorithm selects:

$A \rightarrow B \rightarrow C$

because the cost is **3**, which is lower than 5.

Distance Vector vs Link State

Feature	Distance Vector	Link State
Knowledge	Knows distance through neighbors	Knows complete network topology
Information exchange	With neighbors	Flooded throughout network
Algorithm	Bellman-Ford	Dijkstra
Convergence	Generally slower	Generally faster
Complexity	Simpler	More complex
Memory requirement	Lower	Higher
Example	RIP	OSPF

In simple words

Distance Vector:

"My neighbor says destination C is 2 hops away, so through my neighbor it is 3 hops away."

Link State:

"I have a map of the entire network, so I can calculate the shortest route myself."

Q3. Congestion, Congestion Control, Leaky Bucket, Token Bucket and Fragmentation**1. What is Congestion?**

Network congestion occurs when the amount of data entering a network is greater than the network's ability to process and forward it.

In simple terms:

Too much traffic + insufficient network capacity = congestion

Example:

Imagine a router can process **100 packets/sec**, but suddenly receives **200 packets/sec**.

The router's buffer starts filling up.

Eventually:

- Packets are delayed.
- Packets may be dropped.
- Retransmissions increase.
- Network performance decreases.

2. Why do we need Congestion Control?

Congestion control is required to prevent excessive traffic from overwhelming the network.

It helps to:

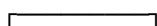
- Reduce packet loss
- Reduce delay
- Improve throughput
- Prevent buffer overflow
- Maintain network stability
- Provide better overall performance

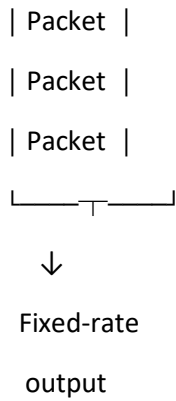
3. Leaky Bucket Algorithm

The **Leaky Bucket** algorithm controls the rate at which packets leave a network.

Imagine a bucket with a small hole at the bottom:

Incoming packets





Packets can enter the bucket at different rates, but they leave at a **fixed rate**.

Example

Suppose:

- Bucket capacity = 10 packets
- Output rate = 2 packets/sec

If 8 packets suddenly arrive, they are stored in the bucket and transmitted at:

2 packets/sec

If more packets arrive and the bucket becomes full, additional packets may be **discarded**.

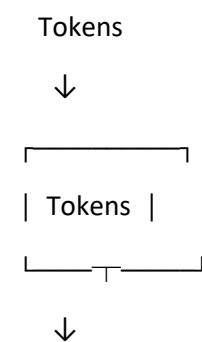
Important characteristic

Leaky Bucket = Fixed output rate

4. Token Bucket Algorithm

In the **Token Bucket** algorithm, tokens are generated at a fixed rate.

A packet needs a token before it can be transmitted.



Packets → Transmission

Suppose:

- Bucket capacity = 10 tokens
- Tokens generated = 2 tokens/sec

If 10 tokens have accumulated, a burst of packets can be transmitted immediately, provided enough tokens are available.

Therefore, token bucket **allows controlled bursts**.

5. Leaky Bucket vs Token Bucket

Feature	Leaky Bucket	Token Bucket
Output	Usually fixed rate	Variable/bursty rate possible
Burst traffic	Does not handle bursts well	Allows bursts
Basic mechanism	Packets wait in bucket	Tokens authorize transmission
If bucket fills	Packets may be discarded	Excess tokens are discarded
Main purpose	Smooth traffic	Control traffic while allowing bursts

Simple example

Suppose a user normally sends:

2 packets/sec

but suddenly needs to send:

8 packets quickly

Leaky Bucket:

Packets are generally released at the fixed rate.

Token Bucket:

If enough tokens are available, the 8 packets can be transmitted as a burst.

6. What is Fragmentation?

Fragmentation is the process of dividing a large IP packet into **smaller packets (fragments)** so that they can pass through a network that has a smaller **MTU (Maximum Transmission Unit)**.

Why is fragmentation needed?

Different networks can have different maximum packet sizes.

For example:

Network A

MTU = 1500 bytes

↓

Router

↓

Network B

MTU = 500 bytes

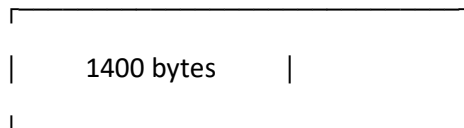
Suppose a packet is **1400 bytes**.

Network B cannot carry the entire 1400-byte packet because its MTU is only 500 bytes.

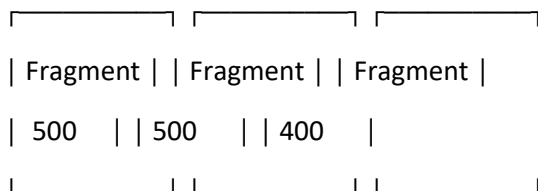
Therefore, the packet may need to be divided into smaller fragments.

Conceptually:

Original packet



Fragmentation



The destination uses information in the IP header to **reassemble the fragments**.

